

Asynchronous FIFO with a Valid/Credit Interface

Round-Trip Latency, Buffer Sizing, Pointer Width, and
Clock-Domain-Crossing Metastability — a Worked Reference

RMC Project – Working Notes

July 5, 2026

Abstract

This note works through, from first principles, why a *credit-based* push interface behaves better than a plain *valid/ready* handshake once the datapath between producer and consumer contains pipeline buffering, how to size the FIFO depth that backs that credit interface, why the internal read/write pointers need one extra bit beyond what indexing the memory strictly requires, and how clock-domain crossing (CDC) and metastability constrain all of the above. Ten fully worked cycle-by-cycle cases build the intuition; closed-form sizing rules and a reference SystemVerilog skeleton close it out. Base async-FIFO structure (dual-pointer, Gray-coded, 2-flop synchronizer) follows the standard construction described at <https://vlsiverify.com/verilog/verilog-codes/asynchronous-fifo/>; this document's contribution is the valid/credit variant layered on top, and the "why," not just the "how."

Contents

1	What we are building	3
1.1	System block diagram	3
1.2	Why a 1-bit pulse, and not the credit <i>count</i> itself?	3
2	Valid/Ready, recap	4
2.1	Why can't <i>ready</i> just stay combinational everywhere?	5
3	The 10 cases	6
4	Depth sizing formulas	12
4.1	Same-clock round trip (Section 3 recap)	12
4.2	Cross-clock round trip (the real async-FIFO case)	12
4.2.1	Worked example – write faster than read (<i>wclk</i> =200 MHz, <i>rclk</i> =100 MHz)	12
4.2.2	Worked example – read faster than write (<i>wclk</i> =100 MHz, <i>rclk</i> =200 MHz)	13
4.2.3	Sizing sweep across clock ratios	13

4.3	Cycle-by-cycle trace: correct depth vs. under-sized depth	14
4.3.1	Correctly sized, $D = 8 \geq \text{RTT} = 6$	14
4.3.2	Under-sized, correctly gated, $D = 1 < \text{RTT} = 6$	14
4.3.3	Under-sized, <i>incorrectly</i> gated (the bug), $D = 1$	15
5	Pointer width: why $[\text{N}:0]$, not $[\text{N}-1:0]$	16
6	Clock domain crossing and metastability	17
6.1	Why crossing domains is dangerous at all	17
6.2	Why a multi-bit bus needs Gray code, not just more sync flops	18
7	Reader behavior: always-ready, sometimes-ready, never-ready	20
7.1	This is not a niche technique	21
8	Frequently confused points	22
9	Summary cheat-sheet	23
10	Reference SystemVerilog skeleton	24
10.1	Closing the gap: the pulse synchronizer	25
10.2	Top-level: wiring the pieces together	26
11	Verification checklist	28
	Glossary	29

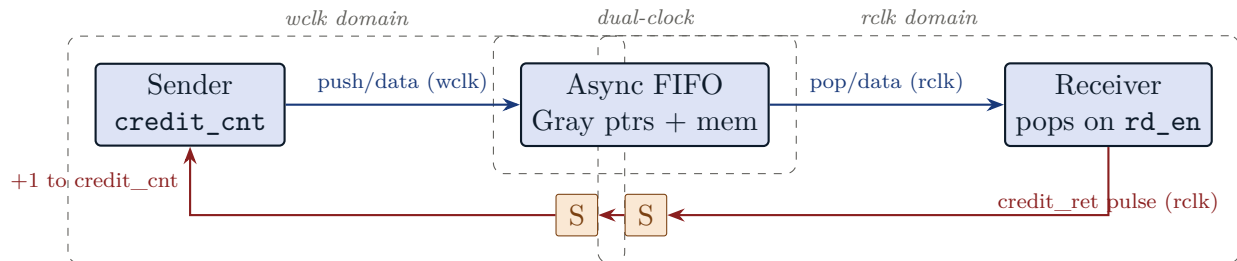
1 What we are building

Two clock domains: a **write domain** (`wclk`) that produces data, and a **read domain** (`rclk`) that consumes it, connected through a dual-clock (asynchronous) FIFO. The interface between them is **not** plain valid/ready. It is **valid/credit**:

- The **sender** (write side) keeps a local `credit_cnt` register. It is allowed to assert `valid` and push a word any cycle `credit_cnt > 0`, decrementing the counter on every accepted push. It never has to wait for a live "may I send" answer.
- The **receiver** (read side), every time it frees up one slot (i.e. every time it pops the FIFO), emits a single-cycle **pulse** — not the credit *count*, not a saturating "credits available" bus, just a 1-bit "I freed one slot" strobe.
- That pulse crosses back into the write domain (through its own CDC synchronizer) and increments `credit_cnt` by 1.

This is the crucial structural difference from valid/ready: **the thing that crosses the round trip is a pulse, and the thing that absorbs the round-trip latency is a counter sized once at design time (the FIFO depth)** — not a live combinational or registered handshake signal that the sender must sample *before* deciding whether to send.

1.1 System block diagram



Forward path (`push/data`, `pop/data`) crosses through the FIFO memory itself, gated by Gray-coded pointers (Section 5). The `credit_ret` pulse crosses back through its own 2-flop synchronizer ("S"-"S" above) directly into the write domain's `credit_cnt` register — it does not go through the FIFO memory at all, it is a side-band 1-bit signal.

1.2 Why a 1-bit pulse, and not the credit *count* itself?

An alternative design would have the receiver compute "credits currently available" as a small bus (say, $\lceil \log_2 D \rceil$ bits) and send that *value* across to the sender, instead of a single accumulating pulse. That alternative is strictly worse here, for reasons that follow directly from Sections 5 and 6:

- A multi-bit value crossing domains has exactly the same torn-sample problem as the pointers do (Section 6): if the available-credits bus changes by more than 1 bit between two samples, and the destination clock samples mid-transition, it can catch a combination of old and new bits that was never a value the counter actually held. Fixing

that would mean Gray-coding the credit-count bus *too* — solving a problem the pulse scheme never creates in the first place.

- **A pulse is inherently single-bit**, so the only CDC primitive it ever needs is the plain 2-flop synchronizer (plus the toggle/edge-detect wrapper for pulse-width safety, Section 10.1) — no Gray coding, no multi-bit comparison logic, nothing.
- **The arithmetic that matters (how many credits are left) stays entirely inside one clock domain** — the write domain's `credit_cnt` register. The read domain never needs to know, or compute, "how many credits are available"; it only needs to notice, locally, "I just freed one slot," which is a fact it already has for free the instant it completes a pop. Moving a *count* across domains would require the read side to maintain its own shadow copy of the write side's arithmetic, doubling the state that has to agree between domains for no benefit.

Key Idea

General principle: whenever a quantity can be re-expressed as "count of discrete, one-at-a-time events" instead of "a computed value," prefer sending the *events* (pulses) across a clock domain and doing the arithmetic locally on one side. It collapses an N -bit CDC problem (which needs Gray coding or an equivalent handshake) into an easier 1-bit CDC problem (which only needs the standard synchronizer). This is precisely the same trick, one level up, as Gray-coding the pointers themselves: Section 5's pointers are also "a count of discrete push/pop events," just exposed as a running total rather than as individual pulses — and note they still need Gray coding for exactly the reason a raw multi-bit credit count would.

2 Valid/Ready, recap

Standard synchronous handshake rules (matches AXI-style valid/ready):

1. A transfer happens on any clock edge where `valid=1` AND `ready=1` are sampled together.
2. Once `valid` is asserted, the sender may **not** withdraw it (or change the data) until a transfer occurs.
3. `ready` is allowed to be a combinational function of `valid` (receiver may say "I can take it, if you're offering") but `valid` must not combinationaly depend on `ready` (no same-cycle "I'll only offer if you can take it" — that creates a combinational loop across the interface).

This works perfectly when sender and receiver are point-to-point with nothing but wires between them. The trouble starts the moment you insert **registers** into either the forward (valid/data) path or the backward (ready) path — which is exactly what happens whenever a pipeline stage, a crossbar hop, or a pipelined memory sits in between. That is the subject of the next section.

2.1 Why can't ready just stay combinational everywhere?

In principle nothing stops you wiring **ready** straight back combinational no matter how many stages sit in the forward path — Case 2 already showed a purely-forward-registered pipeline costs nothing, provided the reverse path is still instantaneous. The reason real designs *can't* keep doing that indefinitely is physical, not logical:

- **Timing closure.** A combinational **ready** that has to travel from the last stage of a deep pipeline all the way back to the first is one long combinational path, competing for the same clock period as every other path in the design. Once that path's delay approaches the clock period, the only fixes are slowing the clock down (usually unacceptable) or registering it (which is exactly what Sections 3 Cases 3–6 analyze).
- **Floorplan distance.** On real silicon the sender and receiver of a handshake are not colocated — a **ready** wire crossing a large block or a NoC hop has real wire delay, plus buffering insertion for signal integrity, both of which show up as pipeline *stages* whether you asked for them or not.
- **Fan-out / fan-in.** A **ready** signal that has to be combinationaly gated by many downstream consumers (e.g. an arbiter with several requestors) accumulates logic depth with every additional consumer, again eating into the same timing budget.

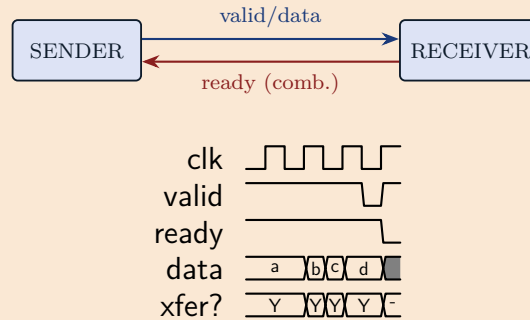
So registered handshake signals are not a design mistake to be avoided — they are a structural consequence of building anything with real pipeline depth or real floorplan distance. The question this document actually answers is: *given* that registers will end up in the round trip somewhere, which handshake discipline degrades gracefully as they accumulate, and which one requires a fresh, local, re-derived buffering decision at every single stage. Section 3 works through both side by side.

3 The 10 cases

Convention used in every waveform below: `clk` is a single shared clock for all of Part 3 (we isolate the CDC problem to Section 6 — mixing both at once obscures which effect is causing what). Each column is one clock cycle, numbered along the top. Shaded cycles mark a *lost* (stalled/bubbled) transfer opportunity.

Case 1 – Valid/Ready, zero buffers (direct wire connect)

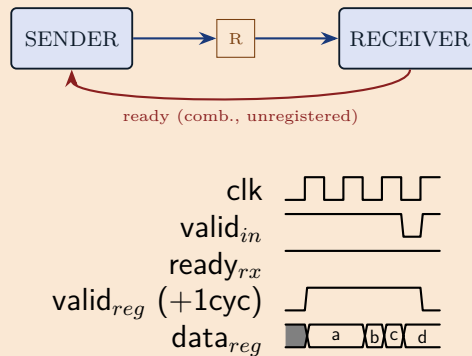
Sender and receiver are directly wired: `valid` and `data` go straight from sender output to receiver input, `ready` comes straight back, both purely combinational (0 gate delay assumed for the diagram).



Round trip: 0 cycles. `ready` reflects the receiver's *true, current* state every cycle because there is no register between "receiver decides" and "sender sees." Every cycle with `valid=1`, `ready=1` transfers. No wasted cycles, no buffering needed. This is the baseline everything else is measured against.

Case 2 – Valid/Ready, 1 register in the forward (data/valid) path only

A pipeline register is inserted *after* the sender, before the receiver, on `valid/data` only. `ready` still runs straight back from receiver to that register's enable (no register on the reverse path).

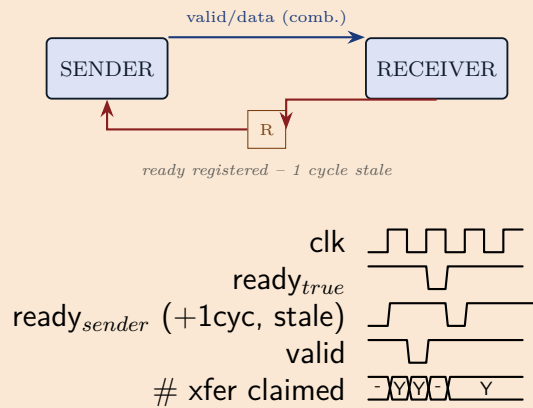


Because `ready` is still combinational from the true receiver state, and the register has an enable tied to that same `ready` (it holds its value if the receiver isn't ready, i.e. it is really

a 1-deep skid register, not a blind pipeline flop), **no throughput is lost** — this case just adds 1 cycle of pure latency, not a round trip. This is the "safe" direction to add a register: forward-path registers only ever cost latency, never bandwidth, *provided* the register has a proper stall/ hold-if-not-ready enable.

Case 3 – Valid/Ready, 1 register in the backward (ready) path only

Now flip it: **valid/data** run straight through combinationally, but the receiver's **ready** is registered before it reaches the sender. This is the dangerous direction.

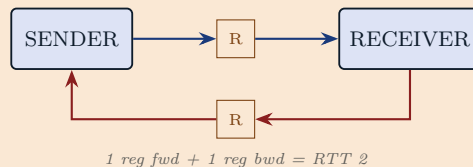


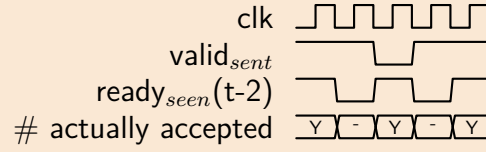
Failure Mode

At cycle 4 the sender is looking at **ready_{sender}**, which is *last cycle's* answer (still high), and sends. But the receiver's *true* state at cycle 4 has already dropped to not-ready. The sender's data is accepted into a receiver that has no room — data corruption/overflow, unless the receiver over-provisions a landing buffer (a skid register) to catch that one "in-flight" beat. **A registered ready signal is only safe if the receiver keeps at least 1 extra slot of storage to absorb whatever was already committed to fly before the stale ready was seen.** This is the first hint that pure valid/ready starts needing extra buffering the moment either direction is registered.

Case 4 – Valid/Ready, 1 register in *both* directions (round trip = 2)

Now both paths are registered: **valid/data** take 1 cycle to arrive, **ready** takes 1 cycle to return. A change in receiver state takes **2 cycles** (1 out, 1 back) to be reflected in what the sender sees.





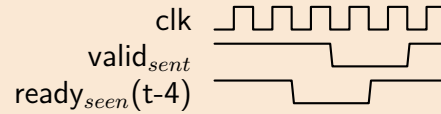
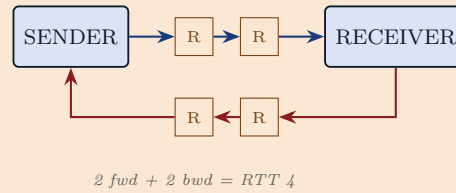
Without any local buffering, every time the receiver's true state flips, the sender is working off information that is 2 cycles stale, so it must either (a) conservatively stop sending the instant it *might* be wrong — losing throughput it didn't need to lose — or (b) risk overrunning like Case 3, now for up to 2 beats in flight instead of 1.

Sizing Rule

To sustain full 1-transfer/cycle throughput safely with a 2-cycle round trip, the receiver-side landing buffer must hold **at least 2** in-flight beats (skid depth = round trip in cycles). This is exactly why registered-ready designs pair every extra pipeline stage with extra skid storage.

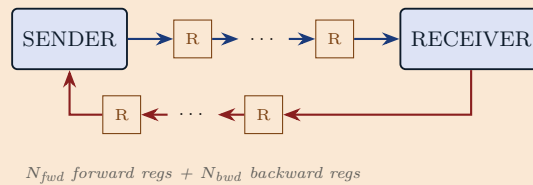
Case 5 – Valid/Ready, 2 registers each direction (round trip = 4)

Same structure, doubled: 2 forward registers, 2 backward registers. Round trip is now $2 + 2 = 4$ cycles.



Skid depth required to not lose throughput: **4**. Notice the pattern: *every* register you add anywhere in either direction directly adds to required buffering elsewhere, linearly, forever. There is no amount of "just add one more pipeline stage for timing closure" that comes for free under valid/ready once you care about sustained throughput.

Case 6 – Valid/Ready, generalized to N registers each direction



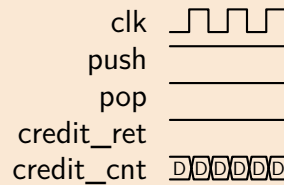
$$\text{RTT}_{\text{cycles}} = N_{\text{fwd}} + N_{\text{bwd}} \quad \text{skid depth to sustain full throughput} \geq \text{RTT}_{\text{cycles}}$$

This is the structural reason valid/ready is disfavored once a datapath has real pipeline depth in it: **every** stage added for timing closure (placement, routing, clock period) must be matched *locally, at that exact stage* by enough skid storage to cover the round trip back to whoever is asserting **valid**. That storage decision is smeared across every stage of the design, has to be re-derived every time the pipeline is re-pipelined, and each skid buffer duplicates state (valid/data storage) that a single, centrally-sized FIFO could have held once. This is the "why not preferred" answer.

Now the credit-based equivalents of the same topologies.

Case 7 – Valid/Credit, zero buffers (baseline, matches Case 1)

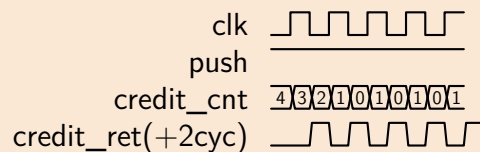
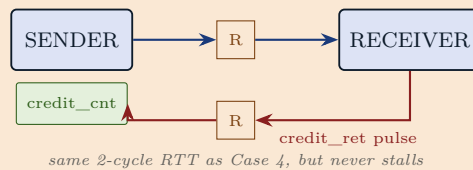
`credit_cnt` initialized to FIFO depth D at reset. Sender pushes whenever `credit_cnt` > 0 ; receiver pulses `credit_ret` every cycle it pops.



With zero physical latency, push and credit-return are same-cycle events — counter never actually moves, it's degenerate, and behaves identically to Case 1. This just establishes the counter's steady state.

Case 8 – Valid/Credit, registers in both directions (round trip = 2, matches Case 4's topology)

Same physical pipeline as Case 4 — 1 register each direction — but now the backward signal is a *credit pulse*, not a live "state" signal, and the sender is driven by its own counter, not by a stale copy of receiver state.



Sender **never stalls** as long as $D \geq \text{RTT}$: it keeps pushing every cycle (draining the pre-loaded counter), and by the time the counter would hit 0 for good, credit pulses from 2 cycles ago are already flowing back to replenish it at the same steady-state rate. **The round trip latency is still physically there** (2 cycles, unchanged) — it just never shows up as a stall, because it was pre-paid for by the initial credit balance (= the FIFO depth) instead of being discovered live.

Case 9 – Valid/Credit, generalized to N registers each direction

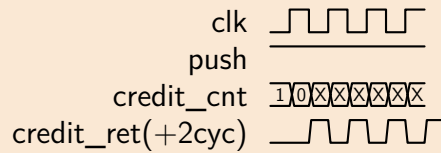
Sizing Rule

$$D_{\min} \geq \text{RTT}_{\text{cycles}} \times (\text{max sustained push rate, in pushes/cycle})$$

For a 1-push-per-cycle producer, $D_{\min} \geq \text{RTT}_{\text{cycles}}$ exactly (Section 4 adds a +1 guard entry and CDC terms on top of this). **Depth absorbs latency; latency never has to be discovered live.** This is why credit interfaces scale to arbitrarily deep pipelines without touching per-stage logic — every extra register anywhere in the round trip just raises one number (required depth) at the one place that already owns storage (the FIFO), instead of demanding a new skid buffer at every stage.

Case 10 – Valid/Credit, *under-sized* depth ($D < \text{RTT}$)

Take Case 8's exact 2-cycle-RTT pipeline, but set $D = 1$ instead of $D \geq 2$.



Failure Mode

At cycle 2, `credit_cnt` is already 0, but `push` is still asserted (nothing told the sender to stop — that *is* the point of a credit scheme, it doesn't ask, it counts). If the counter is allowed to go negative / wrap (marked X above), the sender believes it has credit it doesn't, and the physical FIFO — which really is full — gets written anyway: the write pointer catches and passes the read pointer, **silently overwriting an entry the reader has not consumed yet**. This is the credit-interface equivalent of Case 3's overrun, and it is strictly a *sizing* bug, not a protocol bug: the fix is always $D \geq \text{RTT}$, never "add more handshake logic."

The opposite mistake — over-sizing ($D \gg \text{RTT}$) costs silicon area (deeper memory, wider pointers) but, unlike valid/ready's registered case, **costs zero bandwidth**: the counter just idles higher on average. Contrast this directly with Case 6: in valid/ready, insufficient skid buffering directly steals throughput every cycle it's short; in valid/credit, insufficient depth corrupts data (a much worse, and much more detectable-in-verification, failure) while correct-or-generous depth always gives full throughput. **There is no "wasted bandwidth from oversizing" in a credit scheme** — that penalty is unique to under-buffered valid/ready pipelines, which is precisely why "how much skid do I need" questions disappear once you centralize storage sizing into one FIFO depth number.

4 Depth sizing formulas

4.1 Same-clock round trip (Section 3 recap)

$$D_{\min} = \lceil \text{RTT}_{\text{cycles}} \rceil + 1$$

The +1 is a guard entry: it covers the boundary case where the last credit-consuming push and the first credit-return pulse land on adjacent cycles with no double-counting margin, and is cheap insurance relative to re-deriving exact fencepost arithmetic per design.

4.2 Cross-clock round trip (the real async-FIFO case)

Now the credit pulse must itself cross a clock domain (read domain, where it's generated, back into write domain, where the counter lives), through a standard 2-flop synchronizer (justified in Section 6). Break the round trip into its physical pieces, each measured *in write-clock cycles* since that's the domain the counter lives in and the domain that must not overflow:

Segment	Latency (write-clock cycles)
Write → read domain crossing of the data/valid path (if the write pointer or data itself must be observed in read domain before a pop can happen)	$2 \times T_{rclk}/T_{wclk}$ (2-flop sync, converted)
Any pipeline registers purely inside the read domain before the pop completes	$N_{read_pipe} \times T_{rclk}/T_{wclk}$
Read → write domain crossing of the credit-return pulse (2-flop sync, this time the destination clock <i>is</i> wclk so no conversion needed)	2
Any pipeline registers purely inside the write domain after credit is received	N_{write_pipe}

Sizing Rule

$$D_{\min} \geq \left\lceil 2 \cdot \frac{T_{rclk}}{T_{wclk}} + N_{read_pipe} \cdot \frac{T_{rclk}}{T_{wclk}} + 2 + N_{write_pipe} \right\rceil + 1$$

all evaluated in write-clock cycle units, because the write side is the one whose overflow you must prevent and whose counter is ticking at T_{wclk} .

4.2.1 Worked example – write faster than read (wclk=200 MHz, rclk=100 MHz)

$T_{rclk}/T_{wclk} = 2$. Assume no extra pipe stages either side ($N_{read_pipe} = N_{write_pipe} = 0$):

$$D_{\min} \geq \lceil 2 \cdot 2 + 0 + 2 + 0 \rceil + 1 = 6 + 1 = 7 \Rightarrow \text{round up to } D = 8.$$

Intuition: every single read-domain cycle, up to 2 write-domain cycles’ worth of pushes can occur (write is $2\times$ faster), so any latency measured in read cycles must be inflated by that ratio before it’s counted against the write-side counter. This is *why write-faster- than-read is the sizing-sensitive direction*: the producer can burn through credit much faster than the consumer’s acknowledgements can be generated and relayed back, so the buffer has to be deep enough to cover that whole window at the fast rate.

4.2.2 Worked example – read faster than write (wclk=100 MHz, rclk=200 MHz)

$$T_{rclk}/T_{wclk} = 0.5:$$

$$D_{\min} \geq \lceil 2 \cdot 0.5 + 0 + 2 + 0 \rceil + 1 = 3 + 1 = 4.$$

Intuition: the read domain issues its credit-return pulse (and, if applicable, observes the write pointer) at a rate faster than the producer can consume credit, so the synchronizer latency — while still physically 2–3 read-domain cycles — represents *less* write-domain-cycle "damage window." **You still need the 2-flop synchronizer either way** (metastability doesn’t care about clock ratio), but the depth penalty from that latency shrinks because the write side, which is the slower producer, simply can’t burn through many credits before the faster domain catches up. **Do not skip sizing analysis just because read is faster** — always compute $D_{\min}$ from the formula; it’s smaller here, not absent.

4.2.3 Sizing sweep across clock ratios

Same formula ($N_{read_pipe} = N_{write_pipe} = 0$), swept across common ratios, to show the trend rather than just two endpoints:

wclk	rclk	T_{rclk}/T_{wclk}	raw $D_{\min}$ (pre-round)	final $D_{\min}$
400 MHz	100 MHz	4.0	$2(4) + 2 + 1 = 11$	12 (round to pow2: 16)
266 MHz	100 MHz	2.66	$2(2.66) + 2 + 1 = 8.3$	9 (pow2: 16)
200 MHz	100 MHz	2.0	$2(2) + 2 + 1 = 7$	7 (pow2: 8)
100 MHz	100 MHz	1.0	$2(1) + 2 + 1 = 5$	5 (pow2: 8)
100 MHz	200 MHz	0.5	$2(0.5) + 2 + 1 = 4$	4 (pow2: 4)
100 MHz	400 MHz	0.25	$2(0.25) + 2 + 1 = 3.5$	4 (pow2: 4)

Notice $D_{\min}$ **never drops below ~ 3 – 4** even in the most read-favorable ratio shown — that floor is the synchronizer latency itself (the "2 + 2" fixed cost in the formula, from the two independent 2-flop crossings in each direction), which is a fixed number of cycles regardless of clock ratio. Only the write-faster-than-read term ($2 \cdot T_{rclk}/T_{wclk}$) actually grows with ratio, which is why that direction dominates the sizing decision once the ratio departs from 1.

Key Idea

Rule of thumb: **write-faster-than-read** is the sizing-critical direction (deep buffer, size for it explicitly); **read-faster-than- write** is comparatively forgiving (shallow minimum), but the synchronizer latency itself, and therefore *some* nonzero minimum depth, is never optional in either direction.

4.3 Cycle-by-cycle trace: correct depth vs. under-sized depth

Take the 200/100 MHz worked example: fixed round-trip latency $\text{RTT} = 6$ write-clock cycles between a push and its matching `credit_ret` arriving back (this folds together the read-domain sync, the read-domain pipe, and the write-domain sync from Section 4.2). Sender pushes every cycle whenever `credit_cnt` > 0 . All values below are in write-clock cycles.

4.3.1 Correctly sized, $D = 8 \geq \text{RTT} = 6$

cyc	push?	wptr (dec)	credit_ret?	rprr (dec, =wptr-RTT)	credit_cnt
0	Y	1	–	0	7
1	Y	2	–	0	6
2	Y	3	–	0	5
3	Y	4	–	0	4
4	Y	5	–	0	3
5	Y	6	–	0	2
6	Y	7	Y (from cyc 0)	1	2
7	Y	8	Y (from cyc 1)	2	2
8	Y	9	Y (from cyc 2)	3	2
9	Y	10	Y (from cyc 3)	4	2
10	Y	11	Y (from cyc 4)	5	2

After the initial 6-cycle fill (unavoidable — those credits are what the depth exists to pre-pay for), `credit_cnt` settles at a steady-state floor of $D - \text{RTT} = 8 - 6 = 2$ and **never reaches 0**: push continues every single cycle, forever, at full 1/cycle throughput. This is the "correctly sized" outcome — the floor value (2, here) is your actual margin above the minimum; it is what the "+1 guard / round to next power of 2" step in the formula bought you.

4.3.2 Under-sized, correctly gated, $D = 1 < \text{RTT} = 6$

cyc	push?	wptr (dec)	credit_ret?	credit_cnt	note
0	Y	1	–	0	only credit spent immediately
1	N	1	–	0	stalled – no credit
2	N	1	–	0	stalled
3	N	1	–	0	stalled
4	N	1	–	0	stalled
5	N	1	–	0	stalled
6	Y	2	Y (from cyc 0)	0	credit arrives, spent same cycle
7–11	N	2	–	0	stalled again, 5 more cycles
12	Y	3	Y (from cyc 6)	0	pattern repeats, period 6

No data corruption — `push_gnt` correctly stays low whenever `credit_cnt` = 0 — but throughput has collapsed to **1 push every 6 cycles (1/6 utilization)**. This is the concrete version of the "over-skid/under-depth wastes bandwidth" statement from Case 10: a *correctly*

gated but under-sized credit interface fails safe, at the cost of most of its bandwidth, in direct proportion to how far short of RTT the depth falls.

4.3.3 Under-sized, *incorrectly* gated (the bug), $D = 1$

Now suppose `credit_cnt` is unsigned and the gating check was written wrong (e.g. checked `push_req` only, or the counter is allowed to wrap instead of floor at 0) — a real bug class, not a hypothetical:

cyc	push?	wptr (dec)	phys. addr (=wptr mod $D=1$)	note
0	Y	1	0	data A written to the only slot
1	Y	2	0	data B overwrites A – reader hasn’t even been signa
2	Y	3	0	data C overwrites B, same story

Failure Mode

With physical depth 1 and a 6-cycle round trip, the *very next* push after the first one already lands on the same physical address before the first entry has any chance of being popped — corruption isn’t a rare corner case here, it’s the *first* additional push. This is why Section 4’s formula is a hard floor, not a suggestion: an incorrectly-gated under-sized design doesn’t degrade gracefully at all, it corrupts data on the second beat.

5 Pointer width: why $[N:0]$, not $[N-1:0]$

For a FIFO of depth 2^N entries, indexing the memory only needs N bits ($[N-1:0]$) — that addresses all 2^N locations. But the read/write *pointers* used for full/empty detection are kept **one bit wider**, $[N:0]$, on purpose.

Failure Mode

Why $[N-1:0]$ alone is ambiguous. With only N bits, the write pointer wraps from $2^N - 1$ back to 0 exactly where it started. After a full lap, $\text{wptr} == \text{rptr}$ again — but that's *also* exactly the condition for "FIFO is empty" (no pushes have happened yet). An N -bit-only pointer scheme cannot distinguish "completely full" from "completely empty" — both present as $\text{wptr} == \text{rptr}$.

The fix: widen both pointers to $N+1$ bits. The extra MSB acts as a **wrap/lap counter** — it toggles every time the pointer wraps around the physical memory. The bottom N bits still address the memory normally; the extra bit is only ever compared, never used to index.

$$\text{empty} = (\text{wptr}_{\text{sync}} = \text{rptr}) \quad \text{full} = (\text{wptr}[N] \neq \text{rptr}_{\text{sync}}[N]) \wedge (\text{wptr}[N-1:0] = \text{rptr}_{\text{sync}}[N-1:0])$$

Reading this: *same* bottom bits with the *same* MSB means the read pointer has caught all the way up to the write pointer on the *same lap* — empty. *Same* bottom bits with a *different* MSB means the write pointer has lapped the read pointer exactly once and is now bumping into it from behind — full. That single extra bit is the entire mechanism that disambiguates the two, and it's why every real async FIFO design (including the reference article this note builds on) always carries pointers one bit wider than the address it decodes.

6 Clock domain crossing and metastability

6.1 Why crossing domains is dangerous at all

A flip-flop needs its data input to be stable for a **setup time** before, and a **hold time** after, the active clock edge. A signal generated in a foreign, asynchronous clock domain has *no* relationship to the sampling clock's edges — sooner or later, purely by chance, it will switch *during* that setup/hold window.

When that happens, the flip-flop's internal storage node doesn't cleanly resolve to a valid 0 or 1 in time. It sits balanced near the switching threshold — physically, the two cross-coupled inverters that form the latch are both trying to reinforce a voltage that's ambiguous to both of them. This is **metastability**: an analog voltage sitting near $V_{dd}/2$, not a "wrong digital value," an *undefined* one, that only resolves (settles fully high or fully low) after some — not fixed! — amount of additional time.

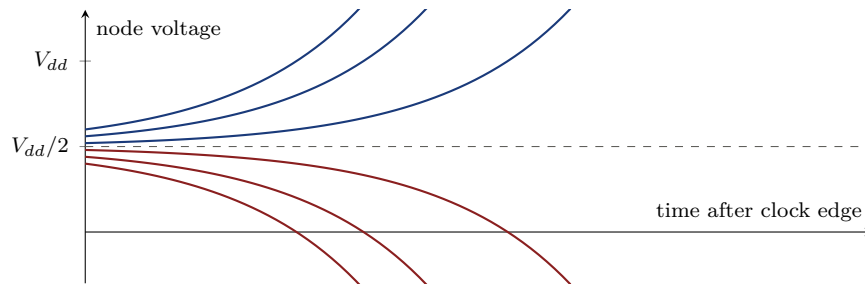
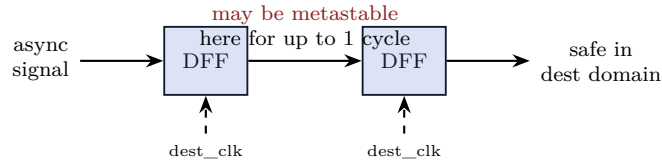


Figure 1: The metastability "bow-tie": trajectories that begin infinitesimally above or below the exact balance threshold diverge *exponentially* in time toward a valid 1 (blue) or 0 (red). Trajectories starting closer to the threshold take longer to resolve; there is no fixed worst-case bound, only a probability that shrinks the longer you wait.

This exponential divergence is exactly why synchronizer design is probabilistic, not deterministic: the **mean time between failures** of a synchronizer stage is

$$\text{MTBF} = \frac{e^{t_r/\tau}}{T_0 \cdot f_{clk} \cdot f_{data}}$$

where t_r is the resolution time you allow (roughly, 1 clock period for a single synchronizer stage), τ is the flip-flop's own metastability time constant (a process/library parameter), T_0 is a device constant, and f_{clk}, f_{data} are the sampling and toggling frequencies. The **exponential** in the numerator is the whole story: **every extra clock period of resolution time you grant improves MTBF exponentially**. That's the entire justification for the standard **2-flop synchronizer**: one flip-flop absorbs the possible metastable event, its output is fed to a *second* flip-flop one clock later, and that one extra period of settling time is (for realistic τ , at real clock speeds) enough to push MTBF from "will glitch, cause system failure" to "longer than the expected life of the universe." A single flip-flop synchronizer is not used in practice specifically because it gives *zero* extra resolution time before that possibly-still- metastable value is used combinatorially or fans out — the second flop is what buys the exponential margin.



Both flops are clocked by the *destination* domain's clock. The first flop is the one allowed to go metastable (its input crossed domains with no timing relationship to `dest_clk`); the second flop samples the first flop's output one full `dest_clk` period later, by which time the exponential settling of Figure 1 has, overwhelmingly, already resolved it to a valid rail.

Key Idea

Noise (thermal, supply, coupling) is exactly what nudges a flip-flop that landed *exactly* on the metastable balance point to one side or the other — that's why the failure mode is sometimes phrased as "noise can flip the bit": it's not corrupting a settled value, it's tipping an *unsettled* one, and which way it tips is genuinely unpredictable from outside the device.

6.2 Why a multi-bit bus needs Gray code, not just more sync flops

Adding a second synchronizer flop fixes metastability *per bit*. It does *not* fix a different, second problem that only shows up on **multi-bit buses**: a plain binary counter can change **many bits at once** on a single increment (e.g. 0111 → 1000 flips all 4 bits). Each of those bits is physically routed on its own wire, with its own independent (if nominally identical) delay, and is sampled by its own independent synchronizer flip-flop pair.

Failure Mode

If the destination-domain clock edge lands *during* that multi-bit transition, different bits can be captured on different sides of their own individual switching point — some already updated, some still showing the old value. The captured word can then be *any* combination of old and new bits, including values the counter **never actually held**, e.g. sampling 0111→1000 mid-flight could yield 1111 or 0000 — a wildly wrong pointer value, not merely a "stale by one" value.

Gray code is the fix: it's a counting sequence in which **exactly one bit changes between any two consecutive values**. With only one bit ever in flight, the worst a torn sample can be is "the old value" or "the new value" — *never* anything else, because there is no third combination available when only 1 bit differs. That's a 1-cycle *ambiguity* (which of the two adjacent, both-valid values did the receiver see), not a *corruption* (a value that was never real). The FIFO's read and write pointers are therefore always converted to Gray code *before* crossing domains (and converted back after), specifically so the multi-bit CDC only ever risks that benign 1-of-two ambiguity.

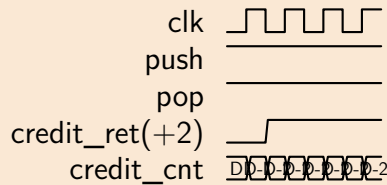
Decimal	Binary	Gray	Bits changed vs. previous
0	000	000	–
1	001	001	1
2	010	011	1
3	011	010	1
4	100	110	1
5	101	111	1
6	110	101	1
7	111	100	1

Conversion is trivial and combinational: $\text{gray} = \text{bin} \oplus (\text{bin} \gg 1)$, so it costs nothing beyond the synchronizer flops already required.

7 Reader behavior: always-ready, sometimes-ready, never-ready

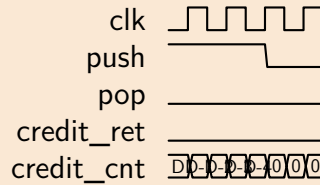
The credit scheme’s job is to make all three of these safe and simple by construction — the sender never special-cases any of them, it only ever watches its own counter.

Reader always ready (drains every cycle it can)



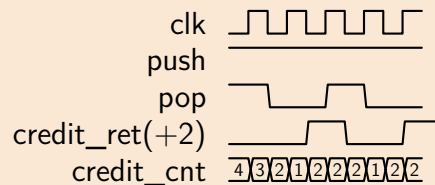
Steady state: counter settles at $D - \text{RTT}$ and holds there, sustaining full 1/cycle throughput indefinitely.

Reader never ready (stalled, e.g. downstream backpressure)



Counter monotonically drains to 0 as the sender keeps pushing with no pops occurring; once it hits 0, **push** correctly deasserts (sender self-throttles). **This is correct, lossless backpressure** — data is never dropped, the sender simply runs out of credit exactly when the physical FIFO runs out of room, because D was sized to make those two events coincide.

Reader sometimes ready (bursty consumption)



Counter free-runs between whatever bursts of pops/returns occur; as long as it never needs to go below 0 (i.e. D was sized for the worst-case burst gap, not just the average rate), the sender never has to know or care that the reader’s cadence is irregular. **This is the real payoff of the credit model**: the sender’s logic is identical in all three of these scenarios; only the counter’s trajectory differs.

7.1 This is not a niche technique

Credit-based flow control under exactly this "sender counts, receiver pulses" shape is the standard mechanism, not a one-off trick invented for this FIFO:

- **PCI Express** link-layer flow control works this way: each side advertises an initial credit allocation per traffic class at link init, the transmitter decrements local credit counters per TLP sent, and the receiver returns credit *update* DLLPs as buffer space frees — structurally identical to Section 1's counter/pulse split, just at the packet-link layer instead of the RTL-clock layer.
- **Network-on-chip (NoC) routers** between IP blocks on the same die almost universally use per-virtual-channel credit counters precisely because the router-to-router hops are pipelined (multi-cycle), making plain valid/ready's round-trip-buffering cost (Case 6) prohibitive at every hop.
- **AXI** itself is valid/ready at the protocol boundary, but any AXI *interconnect* with internal pipelining (which all non-trivial ones have) is typically implemented internally with credit-like skid/store-and-forward buffering precisely to present a compliant valid/ready face at the boundary while not paying Case 6's penalty internally — i.e. the credit discipline often lives just *inside* the block, converted back to valid/ready only at the externally visible ports.

This is worth knowing mainly so the pattern is recognizable later: any time "advertise capacity once, then count pulses" shows up, it's this same tradeoff being made again.

8 Frequently confused points

A handful of questions that tend to come up once the mechanism above makes sense on paper but before it's fully internalized.

"Isn't a credit pulse just a delayed ready?"

No — the difference is not *when* information arrives, it's *what the sender does while waiting for it*. A registered `ready` makes the sender's *next send decision* depend on a signal that is stale by construction; the sender must either stall or risk Case 3/4's overrun. A credit counter makes the sender's send decision depend only on *its own* state, which was correct at reset and is provably kept correct by the invariant "`counter` = physical free slots – in-flight, unacknowledged credit." The pulse's arrival *updates* that local state; it is never consulted live as a permission signal the way `ready` is.

"Why not just make the skid buffer as deep as needed at every stage, and keep valid/ready?"

You could, and Case 6's formula tells you exactly how deep each one needs to be. The credit/FIFO scheme isn't doing anything a correctly-sized per-stage skid buffer chain couldn't do — it is simply *centralizing* that same storage into one place with one size parameter, instead of re-deriving and re-verifying a skid depth at every pipeline stage independently, and instead of paying for N separate small buffers when one buffer of the same total depth does the same job.

"Does the credit scheme make the round-trip latency disappear?"

No. **Latency is physical and unavoidable** — Section 4.3 still shows a full RTT of fill time before steady state, and the very first word pushed still takes RTT cycles of wall-clock time before it's possible for the receiver to signal anything about it back. What the scheme eliminates is *stalling due to not knowing the answer yet* — sustained throughput, not first-word latency.

"What if reset happens with credits already in flight (a push sent, but its matching `credit_ret` hasn't arrived yet)?"

This is exactly why both clock domains' resets, and the synchronizer chains inside `pulse_sync`, must be considered together (Section 11's reset-skew checklist item). The safe pattern is: on any reset of *either* domain, reset the whole FIFO (`wptr`, `rptr`, and every synchronizer flop) back to empty in *both* domains via each domain's own reset, and re-initialize `credit_cnt` to D only after both domains are confirmed out of reset — never assume in-flight credits "settle out" on their own across an asynchronous reset.

"Can `credit_cnt` ever legitimately exceed D ?"

No, and if simulation ever shows it happening, that is a bug, not a corner case: it means a `credit_ret` pulse arrived that doesn't correspond to a push that was ever actually granted (a spurious or duplicated pulse, most likely a `pulse_sync` edge-detection bug). This is exactly the invariant the verification checklist's "credit accounting closure" item (Section 11) is designed to catch.

9 Summary cheat-sheet

	Valid/Ready	Valid/Credit
What crosses the round trip	live receiver state (or a stale registered copy of it)	a single-bit "I freed a slot" pulse
What absorbs latency	nothing, by default – must add matching skid buffers at every registered stage	one counter, pre-loaded with the FIFO depth at reset
Cost of extra pipeline stages	a new skid buffer decision at every stage, every time	one number (depth) goes up, once
Under-sizing failure	throughput loss (safe, just slow) if conservative; data overrun if not	data overrun (write pointer laps read pointer) – must size correctly, no "safe-but-slow" fallback
Over-sizing cost	N/A (you'd just not add the stage)	silicon area only, zero bandwidth cost
Scales to deep pipelines?	poorly – distributed buffering decisions	yes – centralized, one depth parameter

Sizing Rule

Depth sizing, final form:

$$D_{\min} = \left[\underbrace{2 \cdot \frac{T_{rclk}}{T_{wclk}} + N_{read_pipe} \cdot \frac{T_{rclk}}{T_{wclk}}}_{\text{read-domain latency, in wclk units}} + \underbrace{2 + N_{write_pipe}}_{\text{write-domain latency}} \right] + 1$$

Round *up* to the next power of 2 if the pointer/Gray-code scheme requires it. Pointer width = $\lceil \log_2 D \rceil + 1$ bits (the extra bit per Section 5).

10 Reference SystemVerilog skeleton

Structure only – for orienting yourself in an implementation, not a tutorial on how to write SystemVerilog. Standard dual-pointer / Gray-code / 2-flop-synchronizer async FIFO core (per the reference article), with the credit-counter and single-pulse credit-return layered on the write side.

Listing 1: Async FIFO core: memory + Gray pointers + synchronizers

```

1 module async_fifo #(
2     parameter DW = 32,
3     parameter N = 4           // depth = 2**N
4 ) (
5     input  logic          wclk, wrst_n,
6     input  logic          wr_en,
7     input  logic [DW-1:0] wdata,
8     output logic          full,
9
10    input  logic          rclk, rrst_n,
11    input  logic          rd_en,
12    output logic [DW-1:0] rdata,
13    output logic          empty
14 );
15 logic [N:0] wptr_bin, wptr_gray, wptr_gray_rsync1, wptr_gray_rsync2;
16 logic [N:0] rptr_bin, rptr_gray, rptr_gray_wsync1, rptr_gray_wsync2;
17 logic [DW-1:0] mem [0:(1<N)-1];
18
19 // -- write side --
20 always_ff @(posedge wclk or negedge wrst_n)
21     if (!wrst_n) wptr_bin <= '0;
22     else if (wr_en && !full) begin
23         mem[wptr_bin[N-1:0]] <= wdata;
24         wptr_bin <= wptr_bin + 1'b1;
25     end
26 assign wptr_gray = wptr_bin ^ (wptr_bin >> 1); // bin->gray, combinational
27
28 // 2-flop synchronizer, read ptr into write domain
29 always_ff @(posedge wclk or negedge wrst_n)
30     if (!wrst_n) {rptr_gray_wsync2, rptr_gray_wsync1} <= '0;
31     else {rptr_gray_wsync2, rptr_gray_wsync1}
32         <= {rptr_gray_wsync1, rptr_gray};
33
34 assign full = (wptr_gray[N] != rptr_gray_wsync2[N]) &&
35             (wptr_gray[N-1] != rptr_gray_wsync2[N-1]) &&
36             (wptr_gray[N-2:0] == rptr_gray_wsync2[N-2:0]);
37 // (N:N-1 inverted-pair check is one common full-detect form for Gray
38 // pointers; equivalent to the wrap-bit compare shown in Sec. 4)
39
40 // -- read side --
41 always_ff @(posedge rclk or negedge rrst_n)
42     if (!rrst_n) rptr_bin <= '0;
43     else if (rd_en && !empty) rptr_bin <= rptr_bin + 1'b1;
44 assign rptr_gray = rptr_bin ^ (rptr_bin >> 1);
45 assign rdata = mem[rptr_bin[N-1:0]];
46
47 always_ff @(posedge rclk or negedge rrst_n)
48     if (!rrst_n) {wptr_gray_rsync2, wptr_gray_rsync1} <= '0;
49     else {wptr_gray_rsync2, wptr_gray_rsync1}
50         <= {wptr_gray_rsync1, wptr_gray};
51
52 assign empty = (rptr_gray == wptr_gray_rsync2);
53 endmodule

```

Listing 2: Write-side credit counter + read-side credit-return pulse (layered on top of the core above)


```

1 module credit_push #(parameter D = 16)(
2   input  logic wclk, wrst_n,
3   input  logic push_req,      // sender wants to send
4   output logic push_gnt,      // == wr_en into async_fifo
5   input  logic credit_ret_pulse // synchronized, 1-cycle wide, from read side
6 );
7   logic [$clog2(D):0] credit_cnt; // note: extra bit, mirrors Sec. 4 logic
8   assign push_gnt = push_req && (credit_cnt != '0);
9
10  always_ff @(posedge wclk or negedge wrst_n)
11    if (!wrst_n) credit_cnt <= D[$clog2(D):0];
12    else begin
13      unique case ({push_gnt, credit_ret_pulse})
14        2'b10: credit_cnt <= credit_cnt - 1'b1;
15        2'b01: credit_cnt <= credit_cnt + 1'b1;
16        default: credit_cnt <= credit_cnt; // 00: no change, 11: net zero
17      endcase
18    end
19 endmodule
20
21 module credit_return (
22   input  logic rclk, rrst_n,
23   input  logic pop_event,      // 1 cycle, asserted when a pop retires
24   output logic credit_ret_pulse_r // still in rclk domain; sync into wclk
25                                   // with a 2-flop pulse synchronizer
26                                   // (toggle + edge-detect, or a small
27                                   // async FIFO of its own) before use
28                                   // above.
29 );
30   assign credit_ret_pulse_r = pop_event;
31 endmodule

```

Note the 2'b11 case in `credit_push`: a push and a credit-return can legitimately land on the sender's clock in the same cycle (the return pulse arrived from a pop that happened up to RTT cycles ago, unrelated to whatever the sender is doing right now) — net effect on the counter is zero, which is exactly why *simultaneous* alloc+retire must never be treated as "no event," only as "both events, which happen to cancel."

10.1 Closing the gap: the pulse synchronizer

`credit_return` above hands off a single-cycle `rclk` pulse; it cannot be sent through a plain 2-flop synchronizer as-is, because a 1-cycle-wide pulse in a *faster* `rclk` domain can fall entirely inside one `wclk` period and be missed by a slower sampling clock (the classic "pulse too narrow to be seen" CDC bug — unrelated to metastability, and just as fatal). The standard fix is a **toggle-and-resynchronize** pulse synchronizer: convert the pulse to a level (toggle a flip-flop on every pop), synchronize that *level* with an ordinary 2-flop synchronizer (levels, unlike pulses, cannot be "missed" — whatever value it's at, it stays there until the next toggle), then edge-detect the synchronized level back into a clean 1-cycle pulse in the destination domain.

Listing 3: Toggle-based pulse synchronizer: `rclk` pulse -> `wclk` pulse

```

1 module pulse_sync (
2   input  logic src_clk, src_rst_n,
3   input  logic pulse_in,      // 1-cycle wide, src_clk domain
4   input  logic dst_clk, dst_rst_n,
5   output logic pulse_out      // 1-cycle wide, dst_clk domain
6 );
7   logic toggle_src;
8   logic toggle_dst_s1, toggle_dst_s2, toggle_dst_s3;

```

```

9
10 // source domain: pulse -> toggle (a level, safe to synchronize)
11 always_ff @(posedge src_clk or negedge src_rst_n)
12     if (!src_rst_n) toggle_src <= 1'b0;
13     else if (pulse_in) toggle_src <= ~toggle_src;
14
15 // destination domain: 2-flop sync + 1 extra flop for edge detect
16 always_ff @(posedge dst_clk or negedge dst_rst_n)
17     if (!dst_rst_n) {toggle_dst_s3, toggle_dst_s2, toggle_dst_s1} <= '0;
18     else {toggle_dst_s3, toggle_dst_s2, toggle_dst_s1}
19         <= {toggle_dst_s2, toggle_dst_s1, toggle_src};
20
21 assign pulse_out = toggle_dst_s2 ^ toggle_dst_s3; // edge = level changed
22 endmodule

```

This is the piece that actually sits in the "S-S" boxes of the system block diagram in Section 1.1, and it is also the standard building block for crossing the Gray-coded pointers themselves in Section 5 when the design instead chooses to compare pointers directly rather than count discrete credits — the two schemes (direct pointer compare vs. credit count) are two ways of asking the same underlying question ("how much room is left"), and both bottom out in exactly this toggle-synchronize-edge-detect primitive, or the level-based 2-flop synchronizer of Section 6 when what's crossing is already a level (like a Gray pointer bit) rather than a pulse.

10.2 Top-level: wiring the pieces together

The four pieces above (`async_fifo`, `credit_push`, `credit_return`, `pulse_sync`) are never useful in isolation — this is how they compose into the single block from the system diagram in Section 1.1.

Listing 4: Top-level: async FIFO + credit-based push/pop, both domains

```

1 module credit_fifo_top #(
2     parameter DW = 32,
3     parameter N = 3 // depth D = 2*N = 8
4 ) (
5     input logic wclk, wrst_n,
6     input logic push_req,
7     output logic push_gnt,
8     input logic [DW-1:0] wdata,
9
10    input logic rclk, rrst_n,
11    input logic rd_en,
12    output logic [DW-1:0] rdata,
13    output logic rvalid // == !empty, exposed for convenience
14 );
15 localparam int D = (1 << N);
16
17 logic wr_en, full, empty, pop_event, credit_ret_w;
18
19 assign wr_en = push_gnt; // push_gnt already checked credit
20 assign pop_event = rd_en && !empty;
21 assign rvalid = !empty;
22
23 async_fifo #(.DW(DW), .N(N)) u_fifo (
24     .wclk(wclk), .wrst_n(wrst_n), .wr_en(wr_en), .wdata(wdata), .full(full),
25     .rclk(rclk), .rrst_n(rrst_n), .rd_en(rd_en), .rdata(rdata), .empty(empty)
26 );
27
28 credit_push #(.D(D)) u_credit_push (
29     .wclk(wclk), .wrst_n(wrst_n),

```

```

30     .push_req(push_req), .push_gnt(push_gnt),
31     .credit_ret_pulse(credit_ret_w)
32 );
33
34 pulse_sync u_credit_sync (
35     .src_clk(rclk), .src_rst_n(rrst_n), .pulse_in(pop_event),
36     .dst_clk(wclk), .dst_rst_n(wrst_n), .pulse_out(credit_ret_w)
37 );
38 endmodule

```

Two details worth flagging explicitly, since they are the kind of thing that's obvious once pointed out and easy to miss otherwise:

- `full` (from `async_fifo`) is deliberately **unused** by `wr_en` here — the credit counter is the sole authority on whether a push is allowed, by design (Section 1). If `full` and `credit_cnt` ever disagree in simulation, that disagreement — not a wire you'd reach for to "fix" the symptom — is the bug to chase (this is exactly the invariant the verification checklist's "credit accounting closure" item polices).
- `credit_ret_w` both feeds `credit_push` as an input and is driven as `pulse_sync`'s output — a normal same-cycle net, not a combinational loop, because it crosses from `rclk` to `wclk` through registered synchronizer stages; there is no path from `credit_push`'s output back into `pulse_sync`'s input.

11 Verification checklist

Practical checklist for a testbench exercising this design — each item targets one failure mode analyzed above, not generic FIFO sanity.

1. **Max-rate sweep across clock ratios.** Run back-to-back pushes at `credit_cnt` > 0 for every ratio in the sizing sweep table (Section 4), including non-integer ratios — confirm `full` is never asserted incorrectly and no data is dropped.
2. **$D_{\min} - 1$ regression.** Deliberately build the design one entry shallower than the sizing formula recommends and confirm the testbench *catches* the Case 10 overrun (a scoreboard mismatch or an assertion on write-while-full) — if under-sizing doesn't fail visibly in simulation, the checker itself is the bug.
3. **Pulse-width-1 guarantee.** Assert, on both sides of every `pulse_sync` instance, that `pulse_in`/`pulse_out` are never asserted for more than 1 destination-clock cycle, and that back-to-back toggles (two pops on adjacent source cycles) are not merged into one destination pulse or dropped.
4. **Reset behavior.** Assert both `wrst_n` and `rrst_n` independently and skewed in time (real designs don't reset both domains synchronously); confirm pointers, `credit_cnt`, and both synchronizer chains all return to a consistent empty state with no spurious `credit_ret` pulses generated by the toggle flops glitching through reset.
5. **Simultaneous full+empty edge.** Drive push and pop on the exact same cycle when the FIFO has exactly 1 entry — confirm neither `full` nor `empty` glitches, and the Gray-pointer full/empty compare (Section 5) resolves correctly on both domains' next sample.
6. **Gray-code wraparound.** Force the pointers through several full wraps (push/pop counts $\gg 2^N$) and confirm the wrap-bit compare keeps distinguishing full from empty correctly lap after lap — this is the one most likely to silently regress if someone "simplifies" the pointer width back to `[N-1:0]`.
7. **Credit accounting closure.** Over a long random run, assert `pushes - pops = D - credit_cnt` at every cycle (in the write domain) — any divergence means a credit event was double-counted or dropped, independent of whether data corruption is visible yet.
8. **All three reader profiles.** Replay the always-ready, never-ready, and bursty-sometimes-ready scenarios from Section 7 back to back in one run (not three separate tests) to catch state left over from one profile corrupting the next.

Glossary

valid	Sender-asserted signal: "the data on this bus is real, right now."
ready	Receiver-asserted signal: "I can accept a transfer, right now." A transfer occurs exactly when both are 1 on the same clock edge.
credit	A token representing one free slot of storage somewhere downstream; the sender is authorized to send exactly as many outstanding items as it holds credits for.
credit return / credit_ret	The single-cycle pulse the receiver sends upstream each time it frees one slot (typically: each pop).
skid buffer	A small amount of extra storage added at a pipeline stage specifically to hold data that was already committed to move before the stage learned the downstream side wasn't ready.
round-trip time (RTT)	The number of cycles between a sender's action and the earliest cycle that action's consequence could possibly be reflected back to the sender.
CDC	Clock domain crossing: any signal path between two flip-flops clocked by clocks with no fixed phase relationship.
metastability	A flip-flop's internal storage node settling near an indeterminate voltage instead of resolving cleanly to a valid logic level within one clock period, because its setup/hold window was violated.
MTBF	Mean time between (synchronizer) failures — the expected time before a metastable event fails to resolve within the allotted settling time and propagates a bad value.
2-flop synchronizer	Two flip-flops in series, both clocked by the destination domain, used to give a crossing signal one full clock period of settling time before it's used.
Gray code	A binary encoding in which consecutive values differ in exactly one bit, used for any multi-bit counter/pointer that must cross a clock domain.
wrap bit	The extra, $(N+1)^{\text{th}}$, bit carried by FIFO read/write pointers (beyond the N bits needed to index the memory) that disambiguates "just wrapped around" from "never wrapped," and therefore full from empty.
full / empty	FIFO status flags: empty when the (synced) write pointer equals the read pointer including the wrap bit; full when the bottom N bits match but the wrap bits differ.